UNIVERSIDAD TECNOLÓGICA NACIONAL

Tecnicatura Universitaria en Programación
Arquitectura y Sistemas Operativos

Trabajo Práctico Integrador
Virtualización con VirtualBox: instalación de Ubuntu y ejecución de un programa en Python

Profesor: Mauricio Gabriel Pasti
Coordinador: Oscar Alberto Londero
Tutor: Jonathan Zárate
Comisión: 14

María Constanza Maetta –
email:constanzamaetta@gmail.com
8/11/2026

CONTENIDO

desarrollo del trabajo integrador de arquitectura y sistemas operativos 2
1. Introducción 2
2. Marco teórico 2
3. Caso práctico 4
4. Metodología utilizada 5
4.1. Verificación de la imagen ISO de Ubuntu 6
4.2. Creación y configuración de la máquina virtual 6
4.3. Instalación de Ubuntu 8
4.4 Ejecución del programa en Python. 11
4.5. Ejecución del programa en un contenedor Docker 11
4.6. Creación de una instantánea 13
5. resultados obtenidos 13
6. conclusiones 15
7. bibliografía 16
8. anexos 17
Anexo A. Código fuente del programa 17
Anexo B. Dockerfile del programa 17

DESARROLLO DEL TRABAJO INTEGRADOR DE ARQUITECTURA Y SISTEMAS OPERATIVOS

1. INTRODUCCIÓN

En este trabajo se utilizó Oracle VirtualBox para crear una máquina virtual con Ubuntu Desktop 24.04.5 LTS en una computadora con Windows. Se eligió la virtualización porque permite preparar un entorno de práctica con un sistema operativo distinto del anfitrión. En la formación en programación, esto facilita probar aplicaciones y familiarizarse con otras herramientas y sistemas operativos.

El objetivo fue configurar una máquina virtual, instalar Ubuntu y comprobar que un mismo programa en Python podía ejecutarse de dos formas: directamente en el sistema invitado y dentro de un contenedor Docker. Para comparar ambas ejecuciones, se utilizó un programa que calcula el promedio de tres tiempos de respuesta simulados.

2. MARCO TEÓRICO

La virtualización permite crear máquinas virtuales que utilizan los recursos de una computadora física. Cada máquina dispone de recursos virtuales, como memoria RAM, procesadores y almacenamiento, con los que puede ejecutar un sistema operativo y sus aplicaciones. El hipervisor administra el acceso de las máquinas virtuales a los recursos físicos disponibles (Red Hat, 2023).

Existen dos tipos principales de hipervisores. Los de tipo 1 se ejecutan directamente sobre el hardware, como VMware ESXi. Los de tipo 2 funcionan sobre un sistema operativo anfitrión, como Oracle VirtualBox (Red Hat, 2023). En esta práctica se utilizó VirtualBox sobre Windows para ejecutar Ubuntu como sistema operativo invitado.

El sistema anfitrión es aquel sobre el que se instala VirtualBox, mientras que el invitado funciona dentro de la máquina virtual. Esta organización permite utilizar distintos sistemas operativos en una misma computadora y preparar entornos de desarrollo y prueba. Los recursos asignados a las máquinas virtuales están limitados por la capacidad del equipo físico; por eso, al configurarlas, se debe considerar también lo que necesita el anfitrión para continuar funcionando (Oracle, s. f.).

Una máquina virtual puede utilizarse como entorno de pruebas aislado o sandbox: un espacio delimitado para experimentar con instalaciones y configuraciones. Sin embargo, el aislamiento no constituye una garantía absoluta de seguridad; el hipervisor también debe mantenerse protegido, ya que una vulnerabilidad en él puede afectar a las máquinas que administra (Red Hat, 2023).

Docker permite ejecutar aplicaciones en contenedores. Un contenedor es un proceso aislado con los archivos que necesita para funcionar. A diferencia de una máquina virtual, que ejecuta un sistema operativo con su propio kernel, los contenedores comparten el kernel del sistema donde se ejecutan. El kernel es el núcleo del sistema operativo. Esta diferencia permite que los contenedores eviten la necesidad de iniciar un sistema operativo completo para cada aplicación (Docker, s. f.-b).

En este trabajo se combinaron ambas tecnologías: VirtualBox ejecutó Ubuntu como invitado sobre Windows, y Docker funcionó dentro de Ubuntu. Por lo tanto, el contenedor compartió el kernel de Ubuntu y utilizó el intérprete de Python incluido en su imagen. La ejecución directa, en cambio, utilizó el intérprete instalado en Ubuntu.

La imagen Docker proporciona los archivos y la configuración utilizados para crear el contenedor. Para construirla se emplea un Dockerfile, un archivo de texto que describe las instrucciones de construcción. Las instrucciones FROM, WORKDIR, COPY y CMD permiten definir, respectivamente, la imagen base, el directorio de trabajo, los archivos que se incorporan y el comando predeterminado de ejecución (Docker, s. f.-a). En esta práctica, FROM seleccionó la imagen base python:3.12-slim; WORKDIR estableció /app como directorio de trabajo; COPY incorporó tiempos_respuesta.py; y CMD definió el comando para ejecutar el programa al iniciar el contenedor.

El uso del Dockerfile permite documentar la preparación del entorno de la aplicación y facilita repetir su construcción (Docker, s. f.-a). La práctica comparó la ejecución directa del programa en Ubuntu con su ejecución en Docker mediante los mismos datos de entrada. Esta comparación permitió comprobar la coincidencia de los resultados para el caso ensayado; no constituyó una medición de rendimiento entre ambas tecnologías.

INSTANTÁNEAS DE LA MÁQUINA VIRTUAL

Una instantánea o snapshot conserva un estado de la máquina virtual al que se puede regresar posteriormente. Incluye su configuración y el estado de sus discos virtuales; si se toma mientras está funcionando, también guarda el estado de la memoria. Es útil antes de realizar cambios importantes, porque permite recuperar un estado anterior si

los resultados no son los esperados. Al restaurarla, los cambios posteriores en los discos pueden perderse si no se conservan previamente. Una instantánea no reemplaza una copia de seguridad independiente (Oracle, s. f.).

MODOS DE RED

VirtualBox ofrece diferentes modos de conexión según la comunicación que se necesite establecer (Oracle, s. f.):

- NAT: permite que el invitado acceda a redes externas mediante la conectividad del anfitrión; las conexiones entrantes requieren configuraciones adicionales, como el reenvío de puertos.

- Adaptador puente (Bridge): conecta la máquina virtual a la red física como otro equipo de esa red.

- Solo anfitrión (Host-only): permite la comunicación entre el anfitrión y las máquinas virtuales conectadas a esa red, sin proporcionar por sí mismo acceso a Internet.

- Red interna: permite la comunicación entre máquinas virtuales conectadas a la misma red interna, sin incluir directamente al anfitrión ni proporcionar por sí misma acceso a Internet.

En esta práctica se utilizó NAT para instalar paquetes en Ubuntu y descargar imágenes de Docker.

Nivel	Componente de la práctica	Función
Aplicación en contenedor	Python y tiempos_respuesta.py	Ejecutan el programa dentro del contenedor.
Gestión de contenedores	Docker en Ubuntu	Crea y ejecuta el contenedor a partir de la imagen.
Sistema invitado	Ubuntu y su kernel	Proporciona el kernel que comparte el contenedor. También permite ejecutar Python directamente.
Hipervisor	VirtualBox	Proporciona el hardware virtual y administra la máquina virtual.
Sistema anfitrión	Windows	Es el sistema sobre el que se instaló VirtualBox.
Equipo físico	Procesador, RAM y almacenamiento	Aporta los recursos utilizados por todo el entorno.

Tabla. Organización del entorno utilizado en la práctica. Elaboración propia.

3. CASO PRÁCTICO

El caso práctico consistió en preparar un entorno Linux en una computadora con Windows para ejecutar un programa desde la terminal. Para ello, se creó en Oracle

VirtualBox la máquina virtual Ubuntu-UTN-TPI, con 4 GB de memoria RAM, dos procesadores virtuales y un disco virtual de 40 GB. En ese disco se instaló Ubuntu Desktop 24.04.5 LTS.

Para comprobar el funcionamiento del entorno, se utilizó el programa tiempos_respuesta.py, que solicita tres tiempos de respuesta simulados en milisegundos, los guarda en una lista y muestra su promedio con dos decimales. Primero se ejecutó con el intérprete instalado en Ubuntu. Luego se empaquetó en una imagen Docker y se ejecutó en un contenedor dentro de la misma máquina virtual. En ambas pruebas se ingresaron los mismos valores y se compararon las salidas con un cálculo manual.

El repositorio del proyecto se encuentra disponible en: https://github.com/cotimaetta/TPI-Virtualizacion-UTN

4. METODOLOGÍA UTILIZADA

HERRAMIENTAS Y RECURSOS UTILIZADOS

- Oracle VirtualBox 7.2.20 r175154: creación y administración de la máquina virtual.
- Ubuntu Desktop 24.04.5 LTS: sistema operativo invitado.
- Python 3.12.3: ejecución directa del programa dentro de Ubuntu.
- Nano: edición del archivo Python y del Dockerfile.
- Docker 29.1.3: construcción de la imagen y ejecución del programa en un contenedor.
- Windows PowerShell: cálculo de la huella SHA-256 del archivo ISO mediante Get-FileHash.
- GitHub: alojamiento del código, el informe y las evidencias, con registro de cambios mediante commits realizados desde la interfaz web.
- Microsoft Word: elaboración del informe y exportación a PDF.

El programa utilizó funciones incorporadas de Python y no requirió instalar librerías externas.

ORGANIZACIÓN DEL TRABAJO

El trabajo se realizó de manera individual, con autorización de la cátedra. Todas las tareas de configuración, instalación, programación, pruebas, documentación y preparación de la presentación estuvieron a cargo de María Constanza Maetta.

CONSULTA DE FUENTES

Como orientación inicial se utilizó la consigna y el tutorial del docente. Durante el desarrollo y la elaboración del informe se consultó la documentación de las herramientas utilizadas, cuyas referencias se incluyen en la bibliografía.

4.1. VERIFICACIÓN DE LA IMAGEN ISO DE UBUNTU

Antes de crear la máquina virtual, se verificó la integridad de la imagen de instalación ubuntu-24.04.5.1-desktop-amd64.iso. Para ello, se calculó su huella SHA-256 en Windows PowerShell mediante el comando Get-FileHash y se comparó con el valor publicado por Ubuntu. Ambos valores coincidieron, lo que permitió comprobar que el archivo correspondía a la imagen oficial.

La huella obtenida se comparó con el valor publicado por Canonical (s. f.)

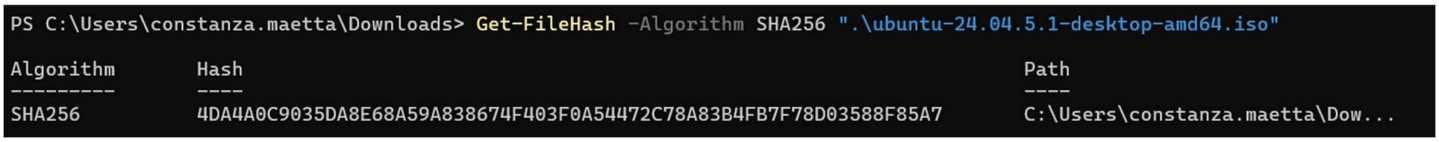


Figura 1. Cálculo de la huella SHA-256 de la imagen ISO de Ubuntu en Windows PowerShell.

4.2. CREACIÓN Y CONFIGURACIÓN DE LA MÁQUINA VIRTUAL

En Oracle VirtualBox se inició la creación de una máquina virtual denominada Ubuntu-UTN-TPI. Se seleccionó el archivo de instalación ubuntu-24.04.5.1-desktop-amd64.iso, verificado en el paso anterior. Se desactivó la instalación desatendida para configurar Ubuntu manualmente y registrar las decisiones tomadas durante el proceso.

Característica	Equipo anfitrión	Máquina virtual
Sistema operativo	Windows de 64 bits	Ubuntu Desktop 24.04.5 LTS de 64 bits

Procesador	Intel Core i7-1165G7 a 2,80 GHz	2 procesadores virtuales (vCPU)
Memoria RAM	40 GB	4 GB (4096 MB)
Almacenamiento	Capacidad de 466 GB; aproximadamente 284 GB libres según la captura inicial	Disco virtual VDI de 40 GB con asignación dinámica
Nombre del equipo o máquina	DESKTOP-QJ25UE3	Ubuntu-UTN-TPI en VirtualBox
Virtualizacion	Habilitada, según el Administrador de tareas de Windows	Administrada mediante VirtualBox

El equipo anfitrión dispone de 40 GB de memoria RAM y un procesador Intel Core i7-1165G7 a 2,80 GHz, con cuatro núcleos y ocho procesadores lógicos. Se comprobó en el Administrador de tareas de Windows que la virtualización estaba habilitada. Considerando estos recursos, se asignaron 4 GB de RAM y dos procesadores virtuales a la máquina virtual para ejecutar Ubuntu y realizar las pruebas con Python y Docker, dejando recursos disponibles para Windows. También se creó un disco virtual VDI de 40 GB con asignación dinámica, cuyo archivo aumenta de tamaño conforme se utiliza, hasta la capacidad configurada.

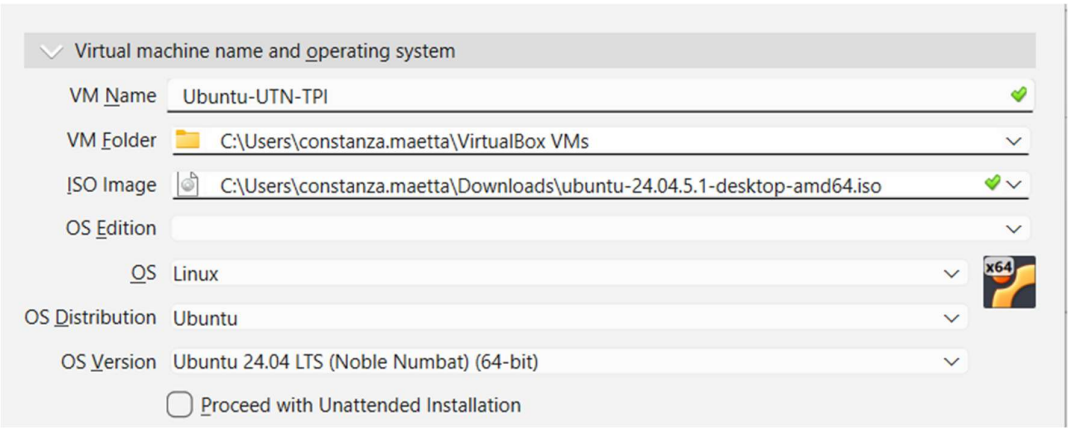


Figura 2. Selección de la imagen ISO y del modo de instalación en VirtualBox.

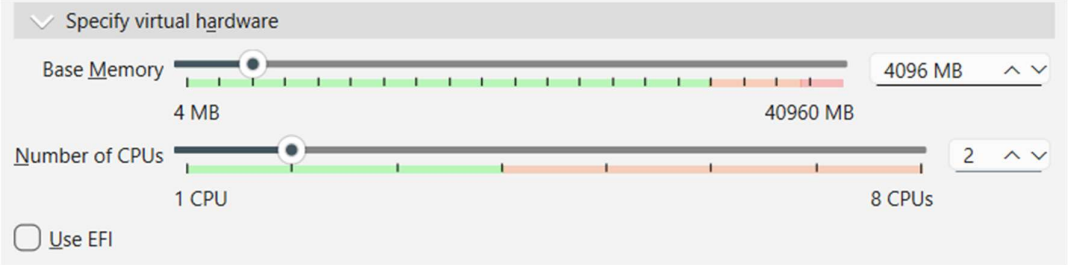


Figura 3. Asignación de memoria RAM y procesadores virtuales en VirtualBox.

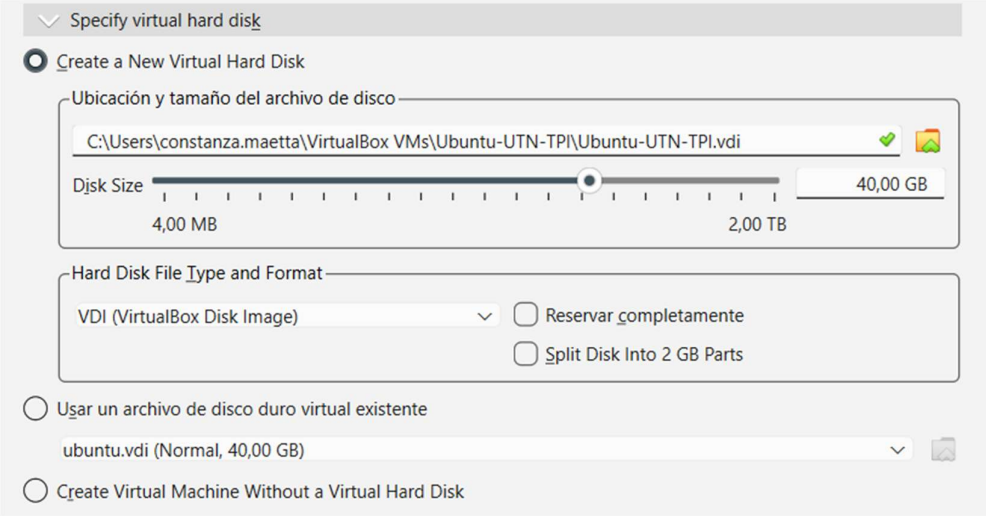


Figura 4. Configuración del disco virtual VDI de 40 GB con asignación dinámica.

4.3. INSTALACIÓN DE UBUNTU

Se inició la máquina virtual desde la imagen ISO verificada. El arranque desde este medio permitió acceder al asistente de instalación de Ubuntu Desktop.

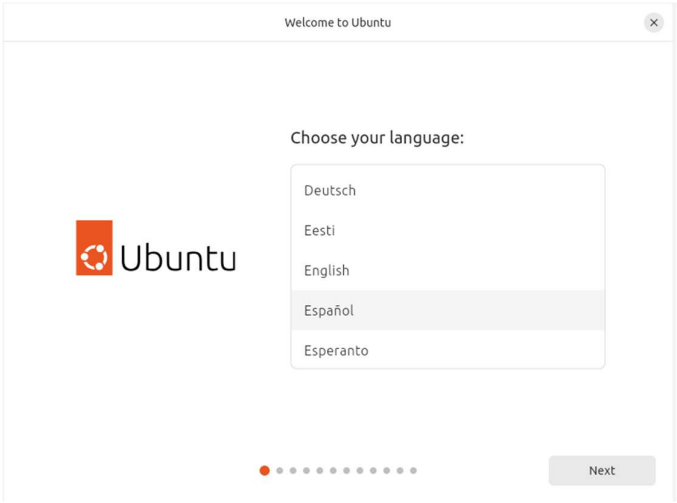


Figura 5. Pantalla inicial del asistente de instalación de Ubuntu Desktop en VirtualBox.

Se continuó la instalación sin conexión a Internet, utilizando la imagen ISO verificada previamente. En el asistente se seleccionó «Instalar Ubuntu» para continuar con la

instalación en el disco virtual. Se eligió la selección predeterminada de aplicaciones porque la práctica se realiza desde la terminal y no requiere una suite de oficina. Se continuó sin instalar software privativo adicional. Se eligió el particionado automático del disco virtual creado para la máquina Ubuntu-UTN-TPI.

Aunque durante la instalación de Ubuntu se seleccionó continuar sin conexión a Internet, la máquina virtual disponía de un adaptador de red configurado en modo NAT, con el cable virtual conectado. NAT permite que el sistema invitado acceda a redes externas utilizando la conectividad del equipo anfitrión. Posteriormente, se utilizó esta conexión para instalar paquetes mediante apt y descargar las imágenes de Docker necesarias para la práctica.

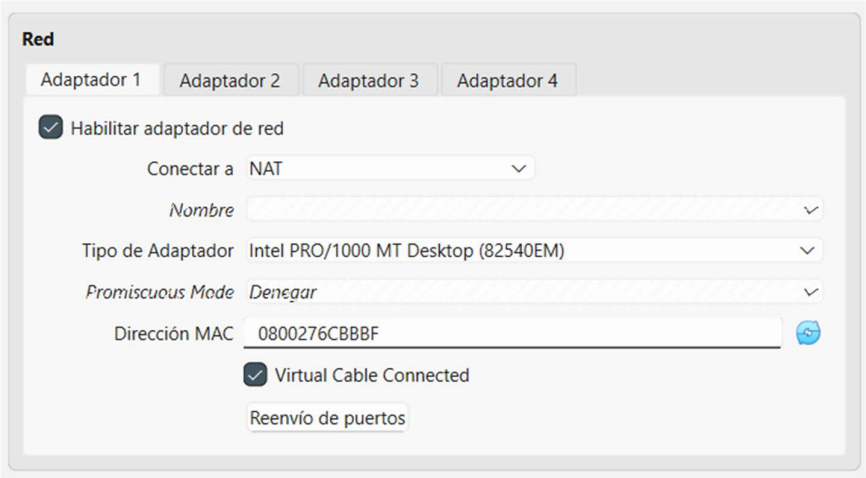


Figura 6. Adaptador de red de la máquina virtual configurado en modo NAT, con el cable virtual conectado.

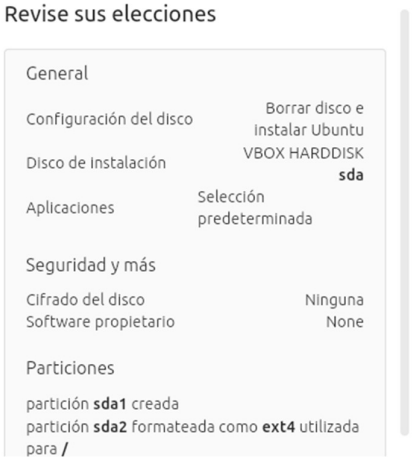


Figura 7. Resumen de instalación de Ubuntu en el disco virtual VBOX HARDDISK (sda).

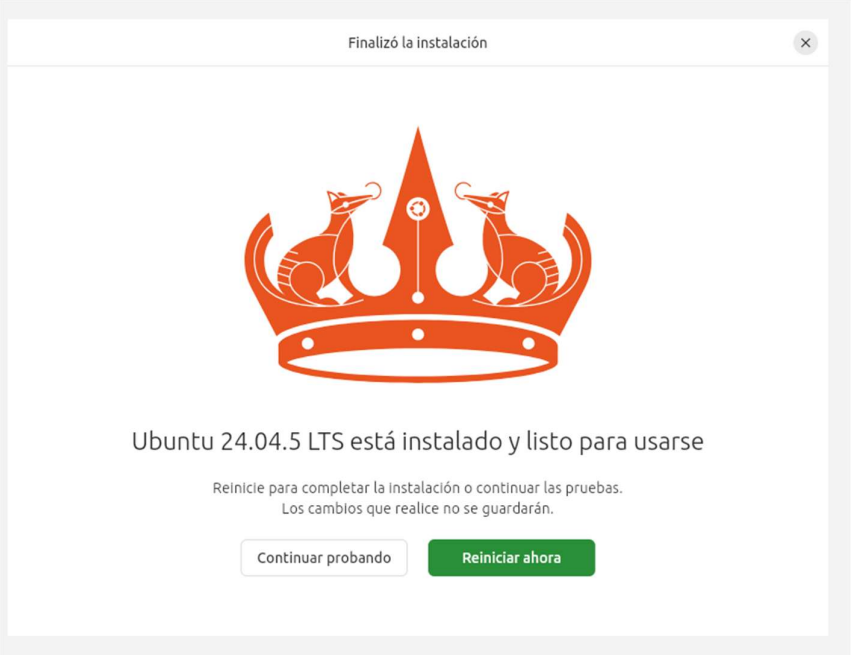
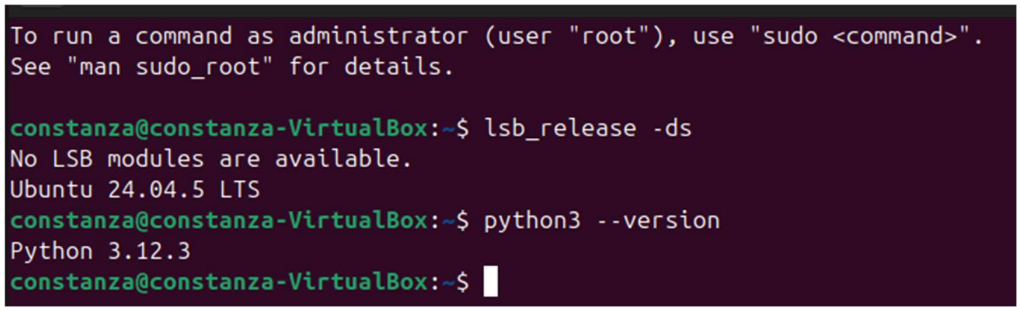


Figura 8. Confirmación de la instalación de Ubuntu 24.04.5 LTS en la máquina virtual.

Tras reiniciar la máquina virtual desde el disco virtual, se inició sesión en Ubuntu. Desde la terminal, se ejecutó lsb_release -ds para verificar que el sistema instalado era Ubuntu 24.04.5 LTS. Luego, con python3 --version, se comprobó que el intérprete



Python 3.12.3 estaba disponible para la práctica.

Figura 9. Verificación de las versiones de Ubuntu y Python en la máquina virtual.

PERSONALIZACIÓN E INTEGRACIÓN DEL ENTORNO

Se utilizó Ubuntu con la interfaz en español y se instalaron las Guest Additions de VirtualBox dentro del sistema invitado. Se configuró el portapapeles compartido y se comprobó que era posible copiar y pegar texto entre Windows y Ubuntu, lo que facilitó el ingreso de comandos durante la práctica.

Como preparación para la instalación, se actualizaron los índices de paquetes y se instalaron las herramientas necesarias mediante los siguientes comandos:

sudo apt update && sudo apt install -y bzip2 tar

sudo apt update && sudo apt install -y build-essential dkms perl linux-headers-$(uname -r)

Después del procedimiento, se reinició Ubuntu con sudo reboot y se verificó el funcionamiento del portapapeles compartido.

4.4 EJECUCIÓN DEL PROGRAMA EN PYTHON.

Se creó el archivo tiempos_respuesta.py con el editor Nano dentro de la máquina virtual. El código fuente completo se incluye en el Anexo A. El programa solicita tres tiempos de respuesta simulados en milisegundos, los almacena en una lista y calcula su promedio. Se ejecutó desde la terminal con python3 tiempos_respuesta.py. Para verificar el cálculo, se ingresaron los valores 120, 150 y 180 ms. El programa mostró un promedio de 150.00 ms, que coincidió con el resultado esperado.



Figura 10. Ejecución del programa en Python y comprobación del promedio con datos simulados.

4.5. EJECUCIÓN DEL PROGRAMA EN UN CONTENEDOR DOCKER

Se instaló Docker en Ubuntu mediante el paquete docker.io. Se verificó la versión instalada con docker --version y se ejecutó sudo docker run --rm hello-world. Docker descargó la imagen de prueba y ejecutó un contenedor que mostró el mensaje «Hello from Docker!».

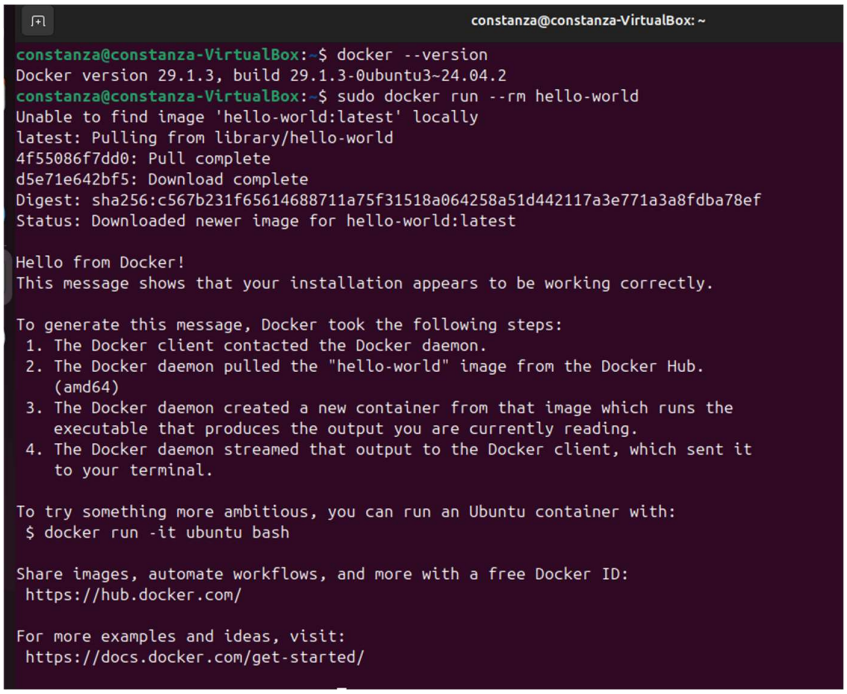


Figura 11. Verificación de Docker 29.1.3 mediante la ejecución del contenedor de prueba hello-world.

Se creó un Dockerfile que utiliza la imagen base python:3.12-slim, copia el archivo tiempos_respuesta.py y define cómo ejecutarlo al iniciar el contenedor. El contenido completo del archivo se incluye en el Anexo B. Se construyó la imagen tiempos-respuesta:1.0 con sudo docker build -t tiempos-respuesta:1.0 . y se verificó su disponibilidad con sudo docker image ls tiempos-respuesta.

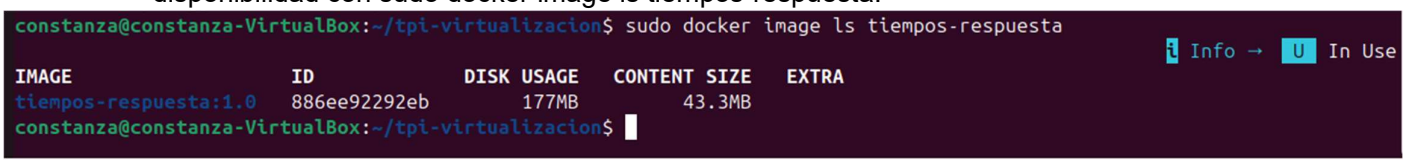


Figura 12. Verificación de la imagen tiempos-respuesta:1.0 tras su construcción.

Se ejecutó el programa en un contenedor creado a partir de la imagen tiempos-respuesta:1.0 mediante el comando sudo docker run --rm -it tiempos-respuesta:1.0. Se ingresaron los mismos tres valores simulados que en la prueba directa en

Ubuntu:120, 50 y 180 ms. El programa mostró un promedio de 150.00 ms, que coincidió con el resultado de la ejecución anterior.

```
constanza@constanza-VirtualBox:~/tpi-virtualizacion$ sudo docker run --rm -it tiempos-respuesta:1.0
Ingresá el tiempo 1 en milisegundos: 120
Ingresá el tiempo 2 en milisegundos: 150
Ingresá el tiempo 3 en milisegundos: 180
Tiempo promedio de respuesta: 150.00 ms
constanza@constanza-VirtualBox:~/tpi-virtualizacion$
```

Figura 13. Ejecución interactiva de tiempos_respuesta.py en un contenedor Docker.

4.6. CREACIÓN DE UNA INSTANTÁNEA

Una vez configurado el entorno, se apagó Ubuntu y se creó en VirtualBox una instantánea denominada «Entorno configurado - Python y Docker». Se conservó así el estado de la máquina virtual con Ubuntu instalado, las Guest Additions, la configuración de red NAT, el programa Python comentado y la imagen Docker actualizada.

Se comprobó que la instantánea aparecía en la lista de VirtualBox. No se realizó una prueba de restauración; su creación dejó disponible un punto al que regresar ante futuras modificaciones del entorno.

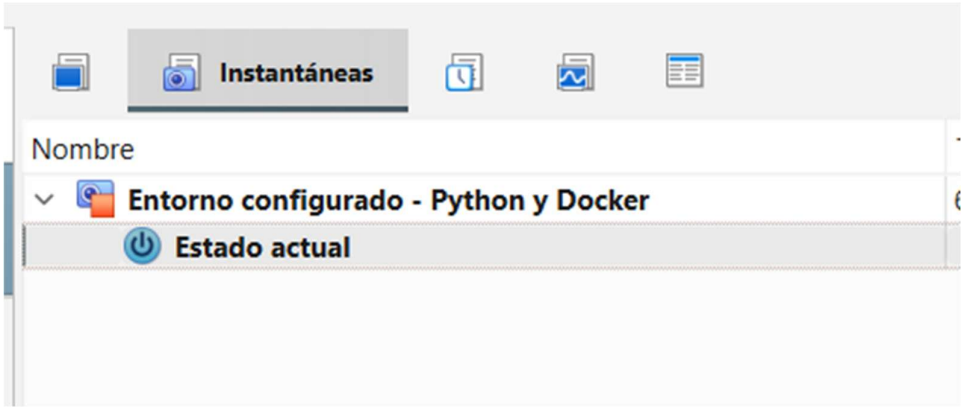


Figura 14. Instantánea «Entorno configurado - Python y Docker» registrada en VirtualBox.

5. RESULTADOS OBTENIDOS

Se instaló Ubuntu 24.04.5 LTS en VirtualBox y se comprobó que la máquina virtual podía iniciar desde su propio disco virtual. Una vez iniciada la sesión, se verificó desde la terminal que Python 3.12.3 estaba disponible (Figura 8).

Se ejecutó tiempos_respuesta.py en dos entornos: directamente en Ubuntu con Python 3.12.3 (Figura 9) y en un contenedor creado a partir de la imagen tiempos-respuesta:1.0 (Figura 12). En ambas ejecuciones se ingresaron los mismos valores simulados, 120, 150 y 180 ms. El programa mostró un promedio de 150.00 ms en los dos casos, que coincidió con el cálculo manual. La comparación verificó que el programa produjo la misma salida en ambos entornos para los datos ensayados.

CASOS DE PRUEBA

Caso	Entradas	Resultado esperado con el código actual	Ubuntu	Docker
1	120, 150, 180	150.00 ms	150.00 ms	150.00 ms
2	10.5, 20.5, 30.5	20.50 ms	20.50 ms	20.50 ms
3	abc como primera entrada	Interrupción con ValueError	ValueError	ValueError

Los casos con números enteros y decimales produjeron los promedios esperados en ambos entornos. Al ingresar «abc», el programa se interrumpió con ValueError al intentar convertir el texto a un número mediante float. Esta prueba permitió identificar una limitación conocida: el programa no controla entradas no numéricas ni solicita su reingreso. Incorporar esa validación se propone como mejora futura.

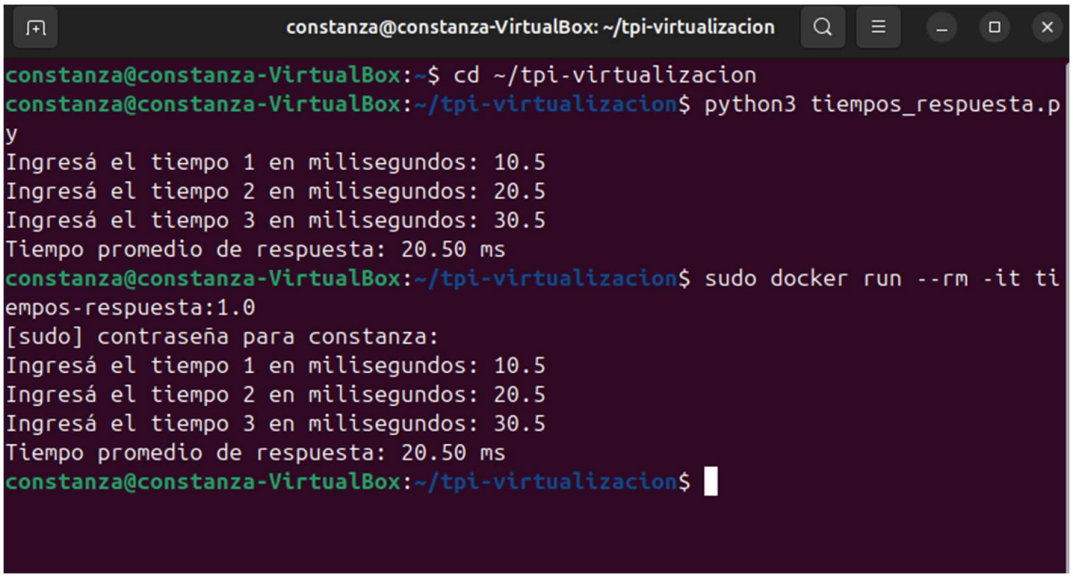


Figura 15. Ejecución con valores decimales en Ubuntu y Docker: promedio obtenido de 20.50 ms.

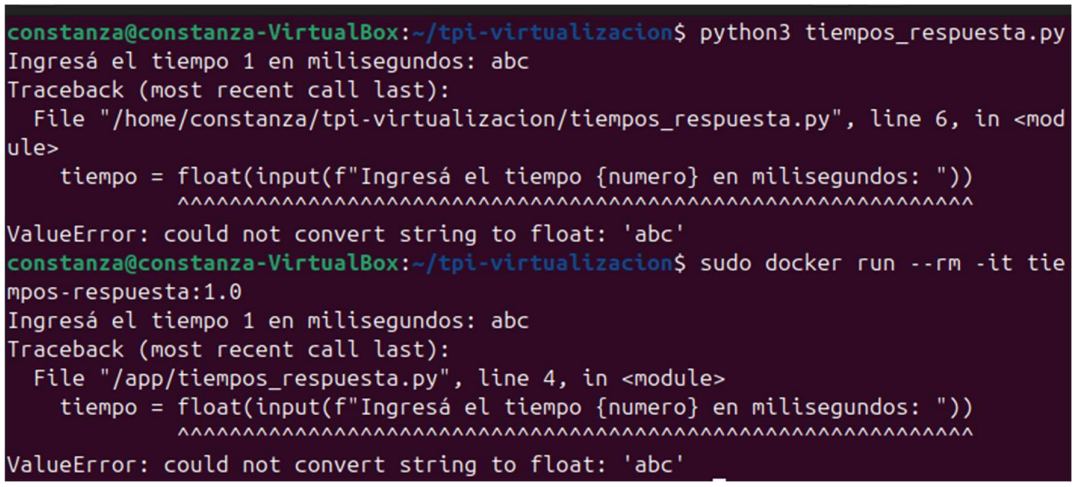


Figura 16. Interrupción con ValueError al ingresar «abc» en Ubuntu y Docker.

6. CONCLUSIONES

Este trabajo me permitió relacionar los conceptos de arquitectura y sistemas operativos con una experiencia práctica. Aprendí a configurar una máquina virtual considerando los recursos del equipo anfitrión, instalar Ubuntu y utilizar su terminal para verificar versiones, instalar herramientas y ejecutar un programa en Python. La asignación de memoria, procesadores virtuales y almacenamiento me ayudó a comprender que la máquina virtual depende de los recursos físicos disponibles y que su configuración debe contemplar también las necesidades de Windows.

La incorporación de Docker me permitió distinguir entre una máquina virtual y un contenedor. VirtualBox proporcionó un entorno con Ubuntu como sistema operativo invitado, mientras que Docker ejecutó la aplicación en un contenedor que compartió el kernel de Ubuntu y utilizó el intérprete de Python incluido en su imagen. Al ejecutar el mismo programa con los mismos datos de entrada, obtuve un promedio de 150.00 ms en ambos casos. Esto comprobó su funcionamiento para el caso ensayado, aunque no constituyó una comparación de rendimiento.

Una de las dificultades fue el arranque inicial de Ubuntu: la pantalla quedó en negro y apareció un mensaje del controlador gráfico vmwgfx. Reiniciar la máquina virtual no resolvió el problema; pude acceder al instalador y completar la instalación al seleccionar «Ubuntu (safe graphics)». También fue necesario preparar el entorno para instalar las Guest Additions y configurar el portapapeles compartido. Después de reiniciar Ubuntu, comprobé que podía copiar y pegar texto entre Windows y el sistema invitado. Estas experiencias me enseñaron a registrar los problemas, realizar cambios concretos y verificar sus resultados antes de continuar.

Entre los beneficios, destaco la posibilidad de practicar con Linux en la misma computadora en la que utilizo Windows, sin necesitar otro equipo físico. El portapapeles compartido facilitó el trabajo con comandos, y el Dockerfile permitió dejar documentada la preparación del entorno de la aplicación. Además, organizar el informe, el código y las capturas en un repositorio facilitó la revisión del procedimiento y la disponibilidad de las instrucciones para repetir las pruebas.

Como mejoras futuras, podría incorporar validaciones al programa para rechazar entradas no numéricas o tiempos negativos y ampliar las pruebas con distintos conjuntos de datos. También podría explorar el uso de instantáneas de la máquina virtual para conservar estados previos a cambios importantes y practicar su restauración. Otra posibilidad sería probar una distribución Linux más liviana o comparar VirtualBox con otras herramientas de virtualización, definiendo previamente qué aspectos evaluar, como facilidad de configuración, compatibilidad y consumo de recursos.

En lo personal, el principal aprendizaje fue comprender mejor qué ocurre en cada nivel del entorno: la computadora física, el sistema anfitrión, la máquina virtual y el contenedor. La práctica también reforzó la importancia de documentar las decisiones y comprobar lo realizado con evidencias, de manera que el trabajo pueda explicarse y reproducirse con claridad.

7. BIBLIOGRAFÍA

Canonical. (s. f.). *SHA256SUMS* [Sumas de comprobación SHA-256 de Ubuntu 24.04 LTS]. Recuperado el 5 de noviembre de 2026, de https://releases.ubuntu.com/24.04/SHA256SUMS

Docker. (s. f.-a). *Dockerfile overview*. Docker Docs. Recuperado el 5 de noviembre de 2026, de https://docs.docker.com/build/concepts/dockerfile/

Docker. (s. f.-b). *What is a container?* Docker Docs. Recuperado el 5 de noviembre de 2026, de https://docs.docker.com/get-started/docker-concepts/the-basics/what-is-a-container/

Microsoft. (s. f.). *Get-FileHash*. Microsoft Learn. Recuperado el 5 de noviembre de 2026, de https://learn.microsoft.com/en-us/powershell/module/microsoft.powershell.utility/get-filehash

Oracle. (s. f.). *Oracle VirtualBox: User guide for release 7.2*. Recuperado el 5 de noviembre de 2026, de https://docs.oracle.com/en/virtualization/virtualbox/7.2/user/EN-VBOX-7-2-USER.pdf

Red Hat. (2023, 3 de enero). *What is a hypervisor?* Recuperado el 5 de noviembre de 2026, de https://www.redhat.com/en/topics/virtualization/what-is-a-hypervisor

8. ANEXOS

ANEXO A. CÓDIGO FUENTE DEL PROGRAMA

A continuación se presenta el contenido del archivo tiempos_respuesta.py, ejecutado con Python 3.12.3 dentro de la máquina virtual. El programa recibe tres tiempos de respuesta simulados en milisegundos y muestra su promedio con dos decimales.

```
# Lista para almacenar los tres tiempos de respuesta simulados.
tiempos = []

# Solicitar tres valores y convertir cada entrada a un número decimal.
for numero in range(1, 4):
    tiempo = float(input(f"Ingresá el tiempo {numero} en milisegundos:
"))

    # Agregar el valor ingresado a la lista.
    tiempos.append(tiempo)

# Calcular el promedio de los tiempos almacenados.
promedio = sum(tiempos) / len(tiempos)

# Mostrar el promedio con dos decimales y su unidad.
print(f"Tiempo promedio de respuesta: {promedio:.2f} ms")
```

ANEXO B. DOCKERFILE DEL PROGRAMA

El siguiente Dockerfile se utilizó para construir la imagen tiempos-respuesta:1.0 a partir de python:3.12-slim.

```
FROM python:3.12-slim
WORKDIR /app
COPY tiempos_respuesta.py .
CMD ["python", "tiempos_respuesta.py"]
```